

Lab Title:E-Commerce Customer Behavior Analysis

Problem Statement:

As a data scientist for an e-commerce company, the goal is to clean and preprocess customer data, which includes information on purchases, demographics, and session activities. The data must be prepared for analysis and machine learning by addressing issues like missing values, duplicates, outliers, and data formatting, ensuring it is structured and ready for modeling.

Objective:

- Data Cleaning: Handle missing values, remove duplicate records, and identify outliers to ensure the dataset is accurate and consistent.
- Data Transformation: Convert unstructured or categorical data into a usable format for machine learning models (e.g., encoding, scaling).
- Feature Engineering: Extract and create relevant features from the raw data to enhance the predictive power of the models.
- Data Structuring: Organize the dataset into a structured format suitable for analysis and model development.
- Prepare Data for Modeling: Ensure the dataset is optimized and ready for training machine learning models that can provide valuable insights for the business.

Requirements:

- Python installed (version 3.x recommended).
- Pandas and NumPy libraries installed (`pip install pandas numpy`).
- Jupyter Notebook (optional but recommended for running the lab).

```
In [2]: # Import necessary library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

Encoding Categorical Data for E-Commerce Customer Behavior Analysis

As a data scientist working for an e-commerce company, you will frequently encounter datasets containing categorical variables like gender, product categories, and customer demographics. These categorical features must be converted into numerical values before they can be used in machine learning models.

Steps Involved:

- Load the Data
- Encoding Categorical Data (One-Hot Encoding & Label Encoding)

- Showcase the Output

```
In [3]: # Sample E-commerce Customer Behavior Data
data = {
    'Customer_id': [101, 201, 301, 401, 501],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'Age': [15, 33, 20, 23, 30],
    'Purchase_amount': [2000, 4050, 2300, 1150, 5500],
    'Product_category': ['Electronics', 'Clothing', 'Electronics', 'Furniture',
    ]
}
```

```
In [4]: # Convert the dictionary to a pandas DataFrame
df = pd.DataFrame(data)
```

```
In [5]: # **Part 1: Encoding Categorical Data for E-Commerce Customer Behavior Analysis**

# 1. One-Hot Encoding for 'gender' and 'product_category'
df_encoded = pd.get_dummies(df, columns=['Gender', 'Product_category'], drop_first=True)

print("Data after One-Hot Encoding:\n", df_encoded)

# 2. Label Encoding for 'gender' (if we want to encode it as numbers instead of labels)
label_encoder = LabelEncoder()

# Apply Label Encoding for 'gender' column
df['gender_encoded'] = label_encoder.fit_transform(df['Gender'])

print("\nData after Label Encoding for 'gender':\n", df[['Customer_id', 'Gender', 'gender_encoded']])
```

Data after One-Hot Encoding:

	Customer_id	Age	Purchase_amount	Gender_Male \
0	101	15	2000	True
1	201	33	4050	False
2	301	20	2300	False
3	401	23	1150	True
4	501	30	5500	False

	Product_category_Electronics	Product_category_Furniture
0	True	False
1	False	False
2	True	False
3	False	True
4	False	False

Data after Label Encoding for 'gender':

	Customer_id	Gender	gender_encoded
0	101	Male	1
1	201	Female	0
2	301	Female	0
3	401	Male	1
4	501	Female	0

```
In [6]: # Import necessary library
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```
In [10]: # **Part 2: Transforming Data for Analysis in E-Commerce Customer Behavior**

# Sample E-commerce Customer Behavior Data
data = {
    'customer_id': [11, 22, 33, 44, 55],
    'gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'age': [25, 30, 22, 35, np.nan], # Missing value in age
    'purchase_amount': [200, 450, np.nan, 150, 500], # Missing value in purchase amount
    'product_category': ['Electronics', 'Clothing', 'Electronics', 'Furniture', 'Electronics'],
    'session_duration': [30, 45, 15, 40, 50],
    'loyalty_score': [7, 9, 6, 5, 8]
}

# Convert to pandas DataFrame
df = pd.DataFrame(data)

# Step 1: Handling Missing Data using SimpleImputer (Mean Imputation)
imputer = SimpleImputer(strategy='mean') # Use mean to fill missing values

# Impute missing values in the 'age' and 'purchase_amount' columns
df['age'] = imputer.fit_transform(df[['age']])
df['purchase_amount'] = imputer.fit_transform(df[['purchase_amount']])

print("Data after Handling Missing Values:\n", df)

# Step 2: Feature Scaling using StandardScaler
scaler = StandardScaler()

# Scale the numerical features: age, purchase_amount, and session_duration
df[['age', 'purchase_amount', 'session_duration']] = scaler.fit_transform(df[['age', 'purchase_amount', 'session_duration']])

print("\nData after Feature Scaling:\n", df)
```

Data after Handling Missing Values:

	customer_id	gender	age	purchase_amount	product_category \
0	1	Male	25.0	200.0	Electronics
1	2	Female	30.0	450.0	Clothing
2	3	Female	22.0	325.0	Electronics
3	4	Male	35.0	150.0	Furniture
4	5	Female	28.0	500.0	Clothing

	session_duration	loyalty_score
0	30	7
1	45	9
2	15	6
3	40	5
4	50	8

Data after Feature Scaling:

	customer_id	gender	age	purchase_amount	product_category \
0	1	Male	-0.677631	-0.919018	Electronics
1	2	Female	0.451754	0.919018	Clothing
2	3	Female	-1.355262	0.000000	Electronics
3	4	Male	1.581139	-1.286626	Furniture
4	5	Female	0.000000	1.286626	Clothing

	session_duration	loyalty_score
0	-0.483494	7
1	0.725241	9
2	-1.692228	6
3	0.322329	5
4	1.128152	8

```
In [11]: # Import necessary library
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
```

```
In [12]: # Sample E-commerce Customer Behavior Data
data = {
    'Customer_id': [101, 201, 301, 401, 501],
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'Age': [15, 33, 20, 23, 30],
    'Purchase_amount': [2000, 4050, 2300, 1150, 5500],
    'Product_category': ['Electronics', 'Clothing', 'Electronics', 'Furniture',
    ]
    # Convert to pandas DataFrame
    df = pd.DataFrame(data)
```

```
In [14]: # **Part 3: Handling Missing Data (Imputation) and Feature Scaling**

# Step 1: Handling Missing Data using SimpleImputer (Mean Imputation)
imputer = SimpleImputer(strategy='mean') # Use mean to fill missing values

# Impute missing values in the 'age' and 'purchase_amount' columns
df['Age'] = imputer.fit_transform(df[['Age']])
df['Purchase_amount'] = imputer.fit_transform(df[['Purchase_amount']])

print("Data after Handling Missing Values:\n", df)

# Feature Scaling
```

```
# Step 2: Feature Scaling using StandardScaler
scaler = StandardScaler()

# Scale the numerical features: age, purchase_amount, and session_duration
df[['Age', 'Purchase_amount']] = scaler.fit_transform(df[['Age', 'Purchase_amount']])

print("\nData after Feature Scaling:\n", df)
```

Data after Handling Missing Values:

	Customer_id	Gender	Age	Purchase_amount	Product_category
0	101	Male	15.0	2000.0	Electronics
1	201	Female	33.0	4050.0	Clothing
2	301	Female	20.0	2300.0	Electronics
3	401	Male	23.0	1150.0	Furniture
4	501	Female	30.0	5500.0	Clothing

Data after Feature Scaling:

	Customer_id	Gender	Age	Purchase_amount	Product_category
0	101	Male	-1.403640	-0.638486	Electronics
1	201	Female	1.342612	0.670410	Clothing
2	301	Female	-0.640792	-0.446940	Electronics
3	401	Male	-0.183083	-1.181199	Furniture
4	501	Female	0.884903	1.596214	Clothing

In [17]: # ***Part 4: Combining and Splitting Datasets for E-Commerce Customer Behavior Analysis*

```
# Steps Involved:
# Combining Datasets : Combine different data sources (e.g., customer information, transaction history, session data)
# Splitting Datasets- Split the combined dataset into training and testing datasets
```

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Sample Customer Demographics Data
demographics_data = {
    'customer_id': [201, 202, 203, 204, 205],
    'gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'age': [44, 20, 42, 35, 60],
}

# Sample Customer Transaction Data
transaction_data = {
    'customer_id': [201, 202, 203, 204, 205],
    'purchase_amount': [200, 450, 300, 150, 500],
    'product_category': ['Electronics', 'Clothing', 'Electronics', 'Furniture', 'Electronics'],
}

# Sample Customer Session Data
session_data = {
    'customer_id': [201, 202, 203, 204, 205],
    'session_duration': [30, 45, 15, 40, 50],
    'loyalty_score': [7, 9, 6, 5, 8]
}

# Convert the dictionaries into pandas DataFrames
df_demographics = pd.DataFrame(demographics_data)
df_transactions = pd.DataFrame(transaction_data)
df_sessions = pd.DataFrame(session_data)
```

```

# Step 1: Combine the Datasets using 'customer_id'
df_combined = pd.merge(df_demographics, df_transactions, on='customer_id', how='inner')
df_combined = pd.merge(df_combined, df_sessions, on='customer_id', how='inner')

print("Combined Dataset:\n", df_combined)

# Step 2: Split the Combined Dataset into Training and Testing Data
X = df_combined.drop(columns=['customer_id', 'loyalty_score']) # Features
y = df_combined['loyalty_score'] # Target variable

# Split the data into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

print("\nTraining Features:\n", X_train)
print("\nTesting Features:\n", X_test)
print("\nTraining Target (Loyalty Score):\n", y_train)
print("\nTesting Target (Loyalty Score):\n", y_test)

```

Combined Dataset:

	customer_id	gender	age	purchase_amount	product_category	\
0	201	Male	44	200	Electronics	
1	202	Female	20	450	Clothing	
2	203	Female	42	300	Electronics	
3	204	Male	35	150	Furniture	
4	205	Female	60	500	Clothing	

	session_duration	loyalty_score
0	30	7
1	45	9
2	15	6
3	40	5
4	50	8

Training Features:

	gender	age	purchase_amount	product_category	session_duration
4	Female	60	500	Clothing	50
2	Female	42	300	Electronics	15
0	Male	44	200	Electronics	30
3	Male	35	150	Furniture	40

Testing Features:

	gender	age	purchase_amount	product_category	session_duration
1	Female	20	450	Clothing	45

Training Target (Loyalty Score):

```

4    8
2    6
0    7
3    5
Name: loyalty_score, dtype: int64

```

Testing Target (Loyalty Score):

```

1    9
Name: loyalty_score, dtype: int64

```

In []: